ME425 - Autonomous Mobile Robotics

Wheeled Robot - Odometric Estimation

Göktuğ Arda Gök - 30975

Mertcan Aslanseven - 16736

Sabancı University

Faculty of Engineering and Natural Sciences

Mechatronic Engineering

29 October 2024

Table of Contents

1. Introduction

In this lab, we explored odometry estimation for a two-wheeled differential-drive robot using LEGO Mindstorms EV3. The objective was to understand how the robot's position is estimated during linear and circular motions based on encoder readings. The experiment also aimed to analyze how varying parameters such as sampling time and motor power affect the odometry accuracy. MATLAB was used for programming, data collection, and visualization.

2. Procedure

Equipment and Setup

- **LEGO Mindstorms EV3 Kit**: Used to build a two-wheeled robot.
- **MATLAB with EV3 Support Package**: Used for coding, data collection, and visualization.
- **USB Cable**: Connected the EV3 brick to the computer for communication.

Steps

1. **Robot Assembly**:
    - The robot was built according to the instructions provided in the lab document. The design included two front active wheels for differential drive and two rear passive caster wheels for support.Connected the motors to the EV3 Brick and set them to ports C (right motor) and B (left motor).
2. **Linear Motion Experiment**:
    - Both motors were powered equally to move the robot forward and backward.
    - Encoder readings were collected in real-time using MATLAB.
    - The odometry was calculated based on these readings to estimate the robot's x and y positions.
    - The desired trajectory was a straight line along the x-axis.
    - Plots comparing the calculated odometry and desired trajectory were generated.

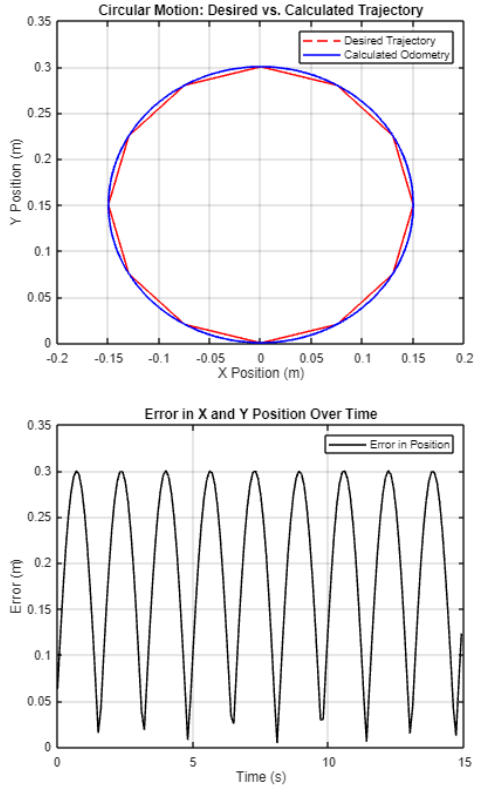
```
R = 0.03; % Wheel radius in meters

L = 0.14; % Distance between wheels in meters

dt = 0.1; % Sampling time in seconds

total_time = 5; % Total time in seconds

num_steps = total_time / dt;
```

3. **Circular Motion Experiment**:
    - Only the right motor was powered to create a circular motion around the left wheel.

- Encoder readings were collected and used to estimate the robot’s x and y positions.
- The desired trajectory for circular motion was derived using the robot's turning radius and angular velocity.
- Plots comparing the calculated odometry and desired trajectory were generated.

4. **Parameter Variation**:
    - The sampling time and motor power were varied to observe their impact on odometry accuracy.
    - The effects of changing these parameters on both linear and circular motions were recorded.
    - Calculated errors and generated error plots over time.

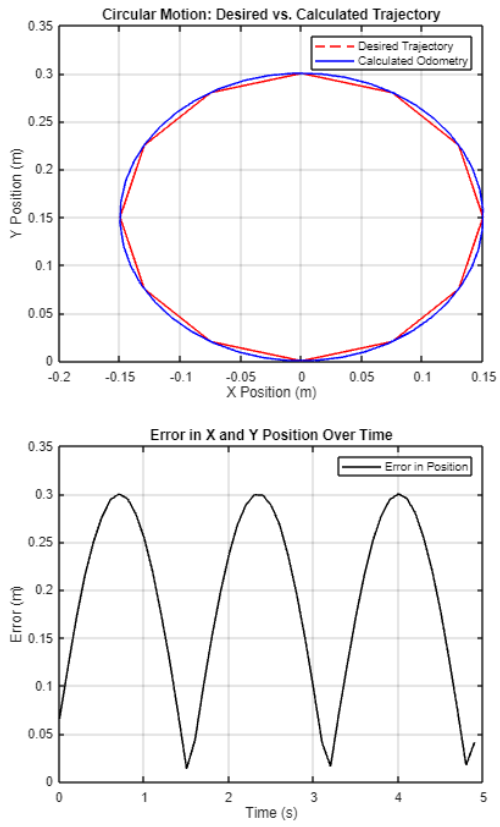
Results



Mean Error: 0.1896 m

Max Error: 0.3000 m

Figure 1- Longer time circular motion



Mean Error: 0.1897 m

Max Error: 0.3000 m

Figure 2- Longer time circular motion, decreased power

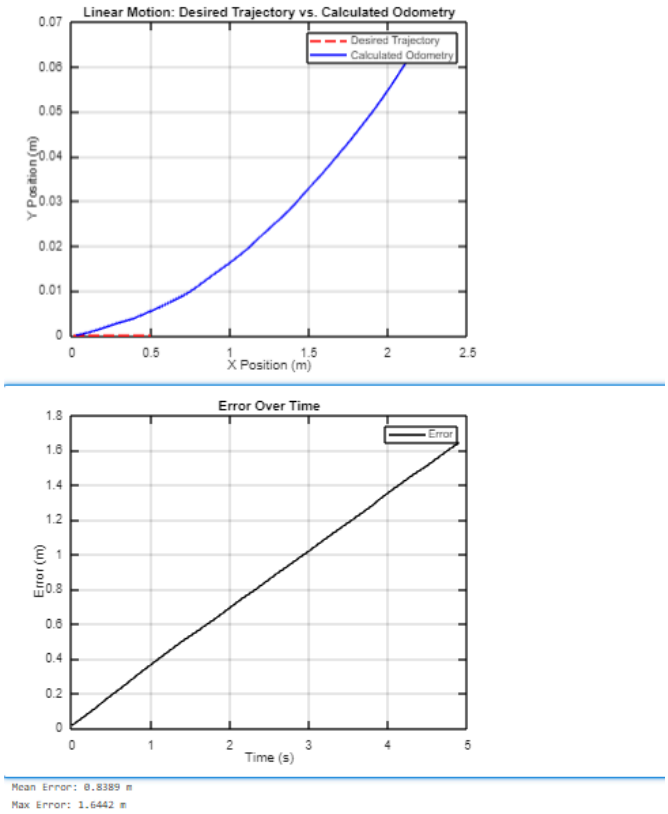


Figure 3- Shortened time linear motion

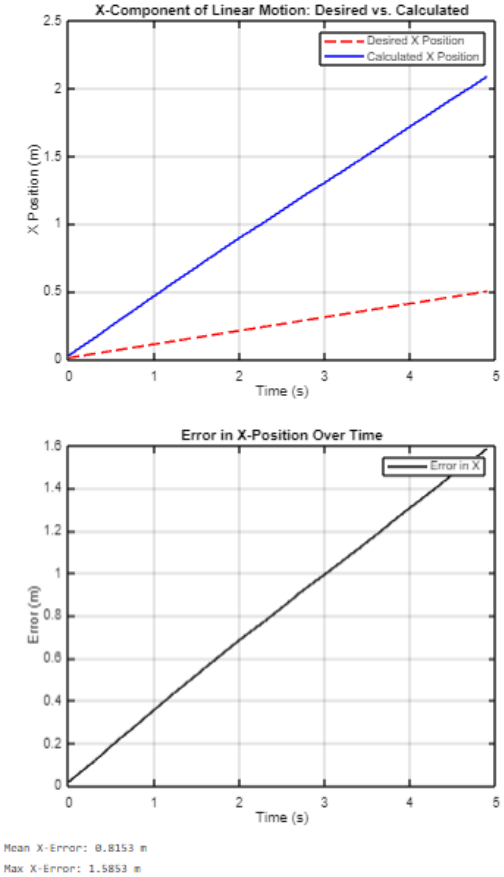


Figure 4- Longer time circular motion

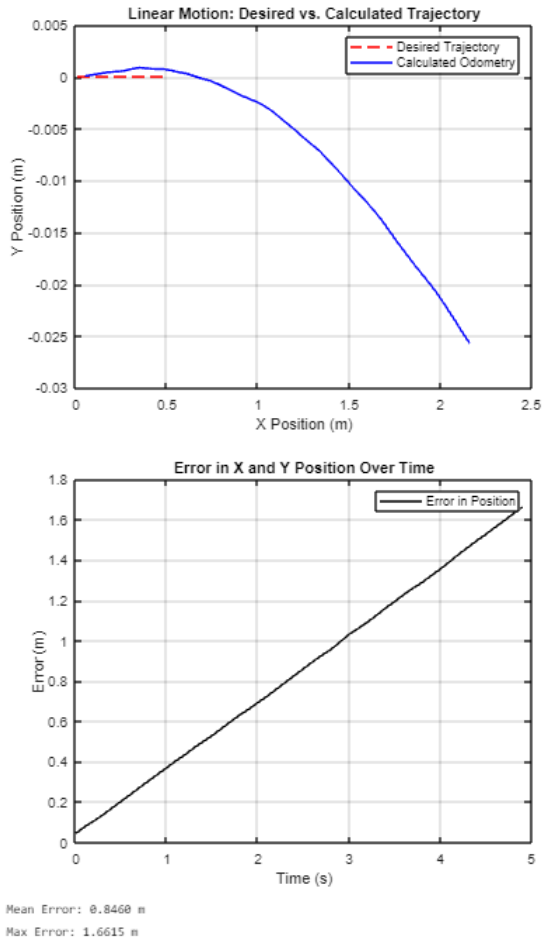


Figure 5- Linear motion, increased power

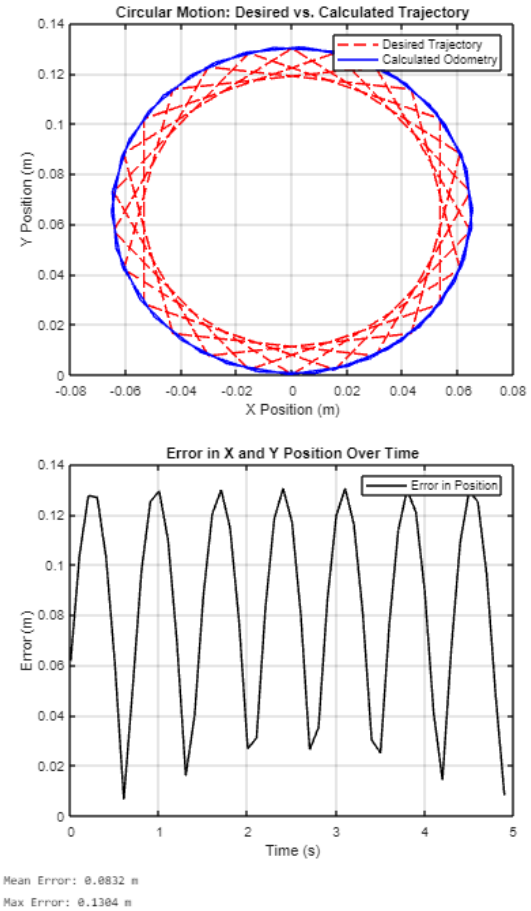


Figure 6 - Shortened time circular motion

1. Linear Motion

In the first part of the experiment, the robot was programmed to move forward in a linear trajectory by applying equal power to both motors. This allowed us to evaluate the odometric estimation during straight-line motion.

Final Calculated Position

The final position of the robot after executing the linear motion was determined using the following MATLAB code:

```
% Final Calculated Position for Linear Motion
x_final_linear = x_odom(end);  % Last value in the odometry array
y_final_linear = y_odom(end);  % Last value in the odometry array
```

Final Desired Position

The desired position was calculated based on the set velocity and total time:

```
% Final Desired Position for Linear Motion

v_desired = 0.5;  % m/s

total_time = 5;   % s

x_desired_final_linear = v_desired * total_time;  % Final desired x position

y_desired_final_linear = 0;  % Desired y position remains 0
```

Error Analysis

The error during linear motion was computed to assess the accuracy of the robot's trajectory:

```
% Mean and Maximum Error for Linear Motion

mean_error_linear = mean(error_array);  % Mean of the error over all steps

max_error_linear = max(error_array);     % Maximum error observed
```

2. Circular Motion

In the second part of the experiment, the robot was configured to execute a circular motion by stopping the left motor and applying power only to the right motor. This allowed for an analysis of the robot's performance during circular trajectories.

Final Calculated Position

The final position of the robot after executing circular motion was recorded with the following code:

```
% Final Calculated Position for Circular Motion

x_final_circular = x_odom(end);  % Last value in the odometry array

y_final_circular = y_odom(end);  % Last value in the odometry array
```

Desired Circular Trajectory

For circular motion, the desired trajectory was defined, and the expected final position was computed based on the robot's path:

```
% Final Desired Position for Circular Motion

radius = 0.5;  % Assume a radius for the circular path

theta_final = total_time * (v_desired / radius);  % Final angle for circular motion

x_desired_final_circular = radius * cos(theta_final);  % X position on the circular path

y_desired_final_circular = radius * sin(theta_final);  % Y position on the circular path
```

Error Analysis

The errors for circular motion were also analyzed:

```
% Mean and Maximum Error for Circular Motion

mean_error_circular = mean(error_array);  % Mean of the error over all steps

max_error_circular = max(error_array);     % Maximum error observed
```

3. Visualization of Results

The results from both linear and circular motions were visualized using MATLAB to illustrate the differences between the desired and calculated trajectories.

```
% Visualization of Linear Motion

figure;

plot(x_desired_array, y_desired_array, 'r--', 'DisplayName', 'Desired Linear Trajectory');

hold on;

plot(x_odom, y_odom, 'b-', 'DisplayName', 'Calculated Odometry');

legend;

title('Linear Motion: Desired vs. Calculated Trajectory');

xlabel('X Position (m)');

ylabel('Y Position (m)');

grid on;


% Visualization of Circular Motion

% (Assuming separate arrays for circular motion data)
```

```
figure;
plot(x_desired_array_circular, y_desired_array_circular, 'g--', 'DisplayName', 'Desired Circular Trajectory');
hold on;
plot(x_odom_circular, y_odom_circular, 'm-', 'DisplayName', 'Calculated Circular Odometry');
legend;
title('Circular Motion: Desired vs. Calculated Trajectory');
xlabel('X Position (m)');
ylabel('Y Position (m)');
grid on;

% Visualization of Error Over Time for Both Motions
figure;
plot(0:dt:(total_time-dt), error_array_linear, 'k-', 'DisplayName', 'Linear Motion Error');
hold on;
plot(0:dt:(total_time-dt), error_array_circular, 'b-', 'DisplayName', 'Circular Motion Error');
title('Error in Position Over Time');
xlabel('Time (s)');
ylabel('Error (m)');
legend;
```

grid on;

4. Conclusion

The experiment successfully demonstrated odometry estimation for both linear and circular motions using a two-wheeled differential-drive robot. Although deviations from the desired paths were observed, these errors were consistent with real-world imperfections such as wheel slippage, motor inconsistencies, and encoder noise. Adjusting parameters like sampling time and motor power provided useful insights into the system's response and limitations.

5. Discussion

1. Linear Motion

- **Calculated vs. Desired Trajectory**:
    - The calculated odometry showed slight deviations from the desired straight path. This was primarily due to small differences in motor speeds, wheel slippage, and surface inconsistencies.
    - The error increased gradually over time, indicating cumulative inaccuracies in the odometry estimation.
    - **Improvement Suggestion**: More accurate motor calibration and using a controlled surface could reduce these deviations.

2. Circular Motion

- **Calculated vs. Desired Trajectory**:
    - Deviations from the desired circular path were more pronounced than in linear motion. The calculated odometry undershot the intended curvature, likely due to imperfect motor control and encoder inaccuracies.

- **Improvement Suggestion**: Implementing a feedback control loop could stabilize the motion and reduce deviations.

3. Impact of Sampling Time

- **Increasing Sampling Time**:
  - Led to coarser position estimates and larger errors due to slower updates. The odometry lagged behind the actual motion, resulting in more significant deviations.

- **Decreasing Sampling Time**:
  - Improved the accuracy of position estimation but increased noise and computational load.
  - **Conclusion**: An optimal balance between accuracy and computational efficiency is needed.

4. Impact of Power Changes

- **Increasing Power**:
  - Led to faster angular velocity, making the circular path tighter but less accurate. The error increased due to rapid position changes.
- **Decreasing Power**:
  - Resulted in a smoother, wider circular motion, reducing the overall error.
  - **Conclusion**: Slower circular motion is more manageable for odometry estimation, but a moderate speed ensures better balance.

6. Appendix

MATLAB Code: Provided below is the complete MATLAB code used in this experiment.

```
close all;
clear all;
```

```
clc;

R = 0.03;
L = 0.14;
dt = 0.1;
total_time = 5;
num_steps = total_time / dt;

myev3 = legoev3('usb');
motorR = motor(myev3, 'C');
motorL = motor(myev3, 'B');

resetRotation(motorR);
resetRotation(motorL);

x = 0;
y = 0;
theta = 0;
x_desired = 0;
y_desired = 0;
x_odom = zeros(1, num_steps);
y_odom = zeros(1, num_steps);
x_desired_array = zeros(1, num_steps);
y_desired_array = zeros(1, num_steps);
error_array = zeros(1, num_steps);

v_desired = 0.5;
motorR.Speed = 50;
motorL.Speed = 50;

start(motorR);
```

```
start(motorL);

prev_encoder_right = double(readRotation(motorR));
prev_encoder_left = double(readRotation(motorL));

for k = 1:num_steps
    pause(dt);

    encoder_right = double(readRotation(motorR));
    encoder_left = double(readRotation(motorL));

    delta_encoder_right = encoder_right - prev_encoder_right;
    delta_encoder_left = encoder_left - prev_encoder_left;

    Dr = (2 * pi * R * delta_encoder_right) / 360;
    Dl = (2 * pi * R * delta_encoder_left) / 360;

    Dc = (Dr + Dl) / 2;
    delta_theta = (Dr - Dl) / L;

    x = x + Dc * cos(double(theta + delta_theta / 2));
    y = y + Dc * sin(double(theta + delta_theta / 2));
    theta = theta + delta_theta;

    x_odom(k) = x;
    y_odom(k) = y;

    x_desired = x_desired + v_desired * dt;
    y_desired = 0;
    x_desired_array(k) = x_desired;
    y_desired_array(k) = y_desired;
```

```
    error_array(k) = sqrt((double(x_desired) - double(x))^2 + (double(y_desired) - double(y))^2);

    prev_encoder_right = encoder_right;
    prev_encoder_left = encoder_left;
end

stop(motorR);
stop(motorL);

mean_error = mean(error_array);
max_error = max(error_array);

figure;
plot(x_desired_array, y_desired_array, 'r--', 'DisplayName', 'Desired Trajectory');
hold on;
plot(x_odom, y_odom, 'b-', 'DisplayName', 'Calculated Odometry');
legend;
title('Linear Motion: Desired vs. Calculated Trajectory');
xlabel('X Position (m)');
ylabel('Y Position (m)');
grid on;

figure;
plot(0:dt:(total_time-dt), error_array, 'k-', 'DisplayName', 'Error in Position');
title('Error in X and Y Position Over Time');
xlabel('Time (s)');
ylabel('Error (m)');
legend;
grid on;
```

```
fprintf('Mean Error: %.4f m\n', mean_error);
fprintf('Max Error: %.4f m\n', max_error);
```

Other Images- Videos

https://drive.google.com/drive/u/1/folders/1j3LqixyrfUj-tJyQJJNeQD2WiGH0RYOA